On the quantum Schwarzschild gravitational theories consisting of field lines, for C++ programmers

S. Halayka*

Wednesday 22nd July, 2026 18:54

Abstract

This paper contains a short introduction to isotropic Schwarzschild gravitation. It is found that with non-normal (e.g. cosine weighted) messenger particle emission comes increased gravitational strength – matching that from the Schwarzschild solution. It is also found that there can be repulsive gravitation. The methods are presented in a tutorial style, and are accompanied by working, multi-threaded C++ code. No prior experience with general relativity is assumed, although some familiarity with Newtonian gravitation and with the C++ language is helpful.

1 Introduction

In [1], we presented a short tutorial for C++ programmers on isotropic Newtonian gravitation. In [1] we built an isotropic gravitational field through the use of pseudorandomly generated field lines. In [1] we used a sphere as the receiver.

In this paper, we find a match between the numerical gravitation and the gravitational time dilation from Schwarzschild's general relativity [2, 3]. Here, we use an axis-aligned bounding box (AABB) as the receiver. The AABB is chosen because the ray-AABB intersection test is one of the simplest and fastest intersection tests available, and because the AABB is the workhorse bounding volume of computer graphics and collision detection. In other words, the tools used in this paper will already be familiar to the practicing C++ graphics programmer.

The core idea is straightforward: a gravitating body (the emitter) is modelled as a sphere that emits a large but finite number of pseudorandom field lines. A small test body (the receiver) is modelled as an AABB placed at some distance from the emitter. Each field line that passes through the receiver contributes one intersecting line segment to the interior of the receiver. The density of these intersecting line segments falls off with distance from the emitter, and it is the *gradient* of this density that plays the role of the gravitational acceleration. Where the density gradient points toward the emitter, gravitation is attractive. Where the density gradient points away from the emitter, gravitation is repulsive.

The single free choice in this model is the *direction* in which each field line is emitted:

- Where the field lines are emitted normal to the surface of the emitter, one obtains Newtonian gravitation.
- Where the field lines are emitted in a cosine weighted fashion about the surface normal, one obtains a stronger gravitation that matches the Schwarzschild solution at close proximity, and that reduces to Newtonian gravitation at far proximity.

This is the main result of the paper: the difference between Newton and Schwarzschild is, in this model, entirely a matter of emission direction statistics.

See Fig. 1 for an example of Newtonian gravitation, where the field lines are normal to the surface of the emitter. See Fig. 2 for an example of Schwarzschild gravitation, where the field lines are not necessarily normal to the surface of the emitter.

In this paper we use the natural Planck units, where $c = G = \hbar = k = 1$. In these units, mass, length, and time are all measured in Planck units, and the equations shed their clutter of physical constants. For instance, the Schwarzschild radius of a body of mass M is simply $r_s = 2M$, and the Newtonian acceleration at distance R is simply $a = M/R^2$.

*sjhalayka@gmail.com

The remainder of the paper is organized as follows. Section 2 reviews the necessary background material: field lines, the Schwarzschild solution, and the holographic entropy bound. Section 3 presents the numerical method in detail. Section 4 discusses the three emission direction sampling strategies. Section 5 presents the C++ code. Section 6 discusses the numerical results, including the onset of repulsive gravitation. Section 7 provides discussion and directions for future work.

2 Background

2.1 Field lines and the inverse square law

The field line picture of gravitation is directly analogous to the field line picture of electrostatics. Consider an isotropic emitter of mass M_e that emits n_e field lines, distributed uniformly over the emitter's surface. By symmetry, at a distance R from the emitter's centre, these n_e field lines pierce a sphere of area $4\pi R^2$. The areal density of field lines is thus:

$$\rho(R) = \frac{n_e}{4\pi R^2}. \tag{1}$$

This is the geometric origin of the inverse square law: the field line count is conserved, while the area over which the field lines are spread grows as R^2. Note that this counting argument holds regardless of whether the field lines are emitted normal to the emitter's surface or not, so long as the emission is statistically isotropic. As we shall see, the direction of emission does not alter the far-field flux, but it does alter the near-field *gradient* statistics, and it is the gradient that we identify with the acceleration.

In the numerical model, we do not have access to a smooth, analytical density function. Instead, we count the number of field lines that intersect a small receiver AABB, and we estimate the density gradient using a finite difference. This makes the model inherently statistical: for a finite field line count n_e, the density estimate carries shot noise, and the finite difference amplifies that noise. Multi-threading and large sample counts are therefore essential, which is why C++ is the implementation language of choice.

2.2 The Schwarzschild solution

The Schwarzschild solution is the unique spherically symmetric vacuum solution of Einstein's field equations [3]. In Schwarzschild coordinates, and in Planck units, the line element is:

$$ds^2 = -\left(1 - \frac{r_e}{R}\right) dt^2 + \left(1 - \frac{r_e}{R}\right)^{-1} dR^2 + R^2 \left(d\theta^2 + \sin^2\theta \, d\phi^2\right), \tag{2}$$

where $r_e = 2M_e$ is the Schwarzschild radius of the emitter. The coefficient of dt^2 encodes the gravitational time dilation: a stationary clock at radius R ticks at the rate

$$\eta = \frac{d\tau}{dt} = \sqrt{1 - \frac{r_e}{R}} \tag{3}$$

relative to a clock at spatial infinity. At the event horizon, where $R = r_e$, the time dilation factor η goes to zero, and the clock appears (to the distant observer) to stop. At far proximity, where $r_e \ll R$, the factor η approaches unity, and the metric reduces to the flat Minkowski metric plus a small Newtonian correction.

It is this time dilation factor, and in particular its radial derivative, that we will match with the numerical field line gravitation. The key qualitative point is that the Schwarzschild acceleration is *stronger* than the Newtonian acceleration at close proximity, diverging at the horizon, while the two agree at far proximity. Any candidate field line model of Schwarzschild gravitation must reproduce this near-field enhancement.

2.3 Entropy and the holographic principle

Why do we tie the field line count to the emitter's surface area? The motivation comes from black hole thermodynamics and the holographic principle [2]. The Bekenstein–Hawking entropy of a black hole is one quarter of the event horizon area, measured in Planck units:

$$S = \frac{A_e}{4}, \tag{4}$$

in nats, or equivalently

$$S = \frac{A_e}{4 \log 2} \tag{5}$$

in bits. In other words, the information content of a black hole scales with its *area*, not with its volume. The holographic principle promotes this observation to a general bound: the maximum entropy that can be contained within a region is proportional to the area of the region's boundary.

In this paper, we take the field lines to be the carriers of this entropy: one field line per bit of horizon entropy. The field line count n_e is therefore fixed by the emitter's Schwarzschild radius, and vice versa. This gives the model a pleasing internal consistency – the strength of the gravitational field, the size of the horizon, and the information content of the emitter are all facets of the same number n_e. It also gives the model its quantum flavour: the field is not a smooth continuum, but a finite collection of discrete lines, and the granularity of the field is set by the Planck scale.

3 Method

3.1 Setting up the emitter and the receiver

Where r_e is the emitter's Schwarzschild radius, r_r is the receiver AABB radius (e.g. half of the AABB side length), and 1e11 and 0.01 are arbitrary constants:

$$r_e = \sqrt{\frac{1\text{e}11 \log(2)}{\pi}}, \tag{6}$$

$$r_r = r_e \times 0.01. \tag{7}$$

The event horizon area is:

$$A_e = 4\pi r_e^2. \tag{8}$$

The entropy (e.g. field line count) is:

$$n_e = \frac{A_e}{4 \log(2)} = 1\text{e}11. \tag{9}$$

Note the order of operations here: we first pick a convenient, computationally tractable field line count $n_e = 1\text{e}11$, and we then solve for the Schwarzschild radius r_e that corresponds to that entropy. The receiver radius r_r is then chosen to be small relative to the emitter, so that the receiver samples the field locally. The receiver must not be too small, however, or else too few field lines will intersect it and the statistics will be poor. The factor of 0.01 is a compromise between locality and statistical quality, arrived at by experimentation.

3.2 Estimating the density gradient

The receiver AABB is centred on the x-axis at distance R from the emitter's centre. A second receiver AABB, of identical size, is centred at distance $R + \epsilon$, where $\epsilon = 2r_r$ so that the two boxes abut without overlapping (see Fig. 2). For each pseudorandom field line, we test for intersection against both boxes, and we accumulate the two intersection counts.

Where R is the distance from the emitter's centre, the derivative is:

$$\alpha = \frac{\beta(R + \epsilon) - \beta(R)}{\epsilon}. \tag{10}$$

Here β is the get intersecting line density function. This is a standard forward finite difference. In the attractive regime, β decreases with distance, α is negative, and the resulting acceleration points toward the emitter. The gradient strength is:

$$g = \frac{-\alpha}{2r_r^3}, \tag{11}$$

where the factor of $2r_r^3$ normalizes by (half) the receiver volume, converting the raw intersection count difference into a proper density gradient.

3.3 Recovering the Newtonian acceleration

From this we can get the Newtonian acceleration a_N, where $r_e \ll R$:

$$a_N = \frac{gR \log 2}{8M_e} = \sqrt{\frac{n_e \log 2}{4\pi R^4}} = \frac{M_e}{R^2}. \tag{12}$$

The middle expression makes the information-theoretic content of the model explicit: the acceleration is set by the field line (bit) count n_e and the distance R, and by nothing else. The right-hand expression is the familiar Newtonian result in Planck units. The chain of equalities holds because the entropy relation $n_e = A_e/(4\log 2) = \pi r_e^2/\log 2$ ties n_e to $r_e = 2M_e$; substituting one for the other converts the bit-counting expression into the mass-based expression. This equality serves as the primary validation of the numerical machinery: before attempting the Schwarzschild case, one should confirm that the code reproduces M_e/R^2 in the normal-emission configuration, to within the expected shot noise.

3.4 Beyond the weak field

We can also get a general relativistic acceleration a_S, where $r_e < R$. In this case, the same gradient-based expression is applied, but is not restricted by the weak-field condition:

$$a_S = \frac{gR\log 2}{8M_e}. \tag{13}$$

That is, the numerical machinery is unchanged – only the field line emission directions change (from normal emission to cosine weighted emission), and the domain of validity extends inward from $r_e \ll R$ to $r_e < R$.

At close proximity, where $r_e \approx R$, the metric produced is related to the Schwarzschild metric – curved space, curved time:

$$\frac{\partial\eta}{\partial R} = \frac{r_e}{2\eta R^2}. \tag{14}$$

$$a_S \approx \frac{\partial\eta}{\partial R}\frac{2}{\pi} = \frac{r_e}{\pi\eta R^2}. \tag{15}$$

The factor of $2/\pi$ is a geometric factor relating the cosine weighted average emission angle to the surface normal. Note that as $R \to r_e$, the time dilation factor $\eta \to 0$, and the acceleration a_S diverges – the numerical model reproduces the infinite surface gravity of the horizon, as measured by a static observer. At far proximity, where $r_e \ll R$, $\eta \approx 1$, and $\frac{\partial\eta}{\partial R} \approx 0$, the metric produced is Newtonian – curved space, practically flat time. In this way, a single numerical model interpolates smoothly between the strong-field Schwarzschild regime and the weak-field Newtonian regime.

4 Sampling the emission direction

The three emission strategies – labelled A), B), and C) in the code of Section 5 – differ only in how the direction $\mathbf{d}$ is drawn. In all three cases, the field line origin is a uniformly distributed pseudorandom point on the emitter's surface.

4.1 A) Normal emission: Newtonian gravitation

In strategy A), the emission direction is simply the surface normal at the origin point:

$$\mathbf{d} = \hat{\mathbf{n}}. \tag{16}$$

Every field line points radially outward, exactly as in the textbook picture of the gravitational (or electrostatic) field of a point mass. This configuration produces Newtonian gravitation, and serves as the validation baseline.

4.2 B) Cosine weighted emission: Schwarzschild gravitation, classical

In strategy B), the emission direction is drawn from a cosine weighted distribution over the hemisphere about the surface normal. The probability density is:

$$p(\theta) = \frac{\cos\theta}{\pi}, \tag{17}$$

where θ is the angle between the emission direction and the surface normal. The mean value of $\cos\theta$ under this distribution is $2/3$, and the mean angle is such that the effective radial component of the emission is reduced by the geometric factor discussed in Section 3. Cosine weighted emission will be familiar to graphics programmers as *Lambertian* emission – it is the emission profile of an ideal diffuse surface. A standard way to generate such a direction is to draw a uniform point on a unit disc, lift it onto the unit hemisphere (Malley's method), and then rotate the hemisphere so that its pole aligns with the surface normal.

4.3 C) Chord emission: Schwarzschild gravitation, quantum

In strategy C), we draw a *second* uniformly distributed pseudorandom point $\mathbf{r}$ on the emitter's surface, and we take the emission direction to be the normalized chord from $\mathbf{r}$ to the origin point:

$$\mathbf{d} = \frac{\mathbf{o} - \mathbf{r}}{\|\mathbf{o} - \mathbf{r}\|}. \tag{18}$$

That is, each field line is the extension of a chord connecting two random points on the horizon. It is a classical result of geometric probability that the direction of such a chord, measured relative to the surface normal at its endpoint, is cosine distributed. Strategies B) and C) therefore produce statistically identical fields.

The distinction is one of interpretation. Strategy B) is a classical prescription: it postulates the cosine weighting as an emission law. Strategy C) is a quantum, exact method: the cosine weighting *emerges* from the geometry of horizon-to-horizon chords, with no distribution postulated by hand. In strategy C), each field line can be read as a messenger particle exchanged between two points on the horizon and then escaping to infinity – which is what motivates the word "quantum" in the title of this paper. The exactness of strategy C) also has a practical benefit: it involves no trigonometric functions and no change of basis, only vector subtraction and normalization.

5 C++ code

The following code uses the Newtonian or Schwarzschild gravitation:
`https://github.com/sjhalayka/schwarzschild_falloff_field_lines`

```
void worker_thread(
        long long unsigned int start_idx,
        long long unsigned int end_idx,
        unsigned int thread_seed,
        const real_type emitter_radius,
        const real_type receiver_distance,
        const real_type receiver_distance_plus,
        const real_type receiver_radius,
        real_type& result_count,
        real_type& result_count_plus)
{
        // Thread-local random number generator
        std::mt19937 local_gen(thread_seed);
        std::uniform_real_distribution<real_type> local_dis(0.0, 1.0);

        real_type local_count = 0;
        real_type local_count_plus = 0;

        // Update progress every N iterations to reduce atomic overhead
        const long long unsigned int progress_update_interval = 10000;
        long long unsigned int local_progress = 0;

        for (long long unsigned int i = start_idx; i < end_idx; i++)
        {
                vector_3 location = random_unit_vector(local_gen, local_dis);
                location.x *= emitter_radius;
                location.y *= emitter_radius;
                location.z *= emitter_radius;

                vector_3 surface_normal = location;
                surface_normal.normalize();


                // A) Newtonian gravitation
                //vector_3 normal =
                //      surface_normal;

                // B) Schwarzschild gravitation, classical
                //vector_3 normal =
                //      random_cosine_weighted_hemisphere(
                //              surface_normal, local_gen, local_dis);

                // C) Schwarzschild gravitation, quantum
                vector_3 r = random_unit_vector(local_gen, local_dis);
```

```
                r.x *= emitter_radius;
                r.y *= emitter_radius;
                r.z *= emitter_radius;
                vector_3 normal = (location - r).normalize();


                local_count += intersect(
                        location, normal,
                        receiver_distance, receiver_radius);

                local_count_plus += intersect(
                        location, normal,
                        receiver_distance_plus, receiver_radius);

                // Update global progress periodically
                local_progress++;
                if (local_progress >= progress_update_interval)
                {
                        global_progress.fetch_add(
                                local_progress, std::memory_order_relaxed);

                        local_progress = 0;
                }
        }

        // Add any remaining progress
        if (local_progress > 0)
        {
                global_progress.fetch_add(
                        local_progress, std::memory_order_relaxed);
        }

        result_count = local_count;
        result_count_plus = local_count_plus;
}
```

Here we see that there are at least 2 different ways of looking at the same Schwarzschild solution. That is, solution B) uses a cosine weighted approximation method. Solution C) uses a quantum, exact method.

Note that the code is highly parallelizable, and so it takes advantage of C++ standard multi-threading. Each worker thread owns its own `std::mt19937` pseudorandom number generator, seeded uniquely, so that no locking is required in the inner loop. The only shared state is the atomic progress counter, which is updated in batches of 10000 iterations to keep the atomic overhead negligible. The per-thread intersection counts are written to caller-owned accumulators only once, after the loop completes, and are summed by the main thread after `join()`. On a machine with T hardware threads, the speedup is very nearly a factor of T, since the workload is embarrassingly parallel and memory traffic is minimal.

5.1 Supporting functions

For the reader who wishes to follow along without immediately consulting the repository, we sketch the supporting functions used by the worker thread. The uniformly distributed pseudorandom unit vector can be generated by drawing the z component uniformly on $[-1, 1]$ and the azimuth uniformly on $[0, 2\pi)$:

```
vector_3 random_unit_vector(
        std::mt19937& local_gen,
        std::uniform_real_distribution<real_type>& local_dis)
{
        const real_type z = local_dis(local_gen) * 2.0 - 1.0;
        const real_type a = local_dis(local_gen) * 2.0 * pi;

        const real_type r = sqrt(1.0f - z * z);
        const real_type x = r * cos(a);
        const real_type y = r * sin(a);

        return vector_3(x, y, z).normalize();
}
```

The cosine weighted hemisphere sample (used by strategy B) can be generated by Malley's method – draw a uniform sample on the unit disc, then project it up onto the hemisphere – followed by a change of basis into the

frame of the surface normal:

```
vector_3 random_cosine_weighted_hemisphere(
        vector_3 normal,
        std::mt19937& local_gen,
        std::uniform_real_distribution<real_type>& local_dis)
{
        vector_3 r = vector_3(local_dis(local_gen), local_dis(local_gen), 0.0);
        vector_3 uu = normal.cross(vector_3(0.0, 1.0, 1.0)).normalize();
        vector_3 vv = uu.cross(normal);

        double ra = sqrt(r.y);
        double rx = ra * cos(2.0 * pi * r.x);
        double ry = ra * sin(2.0 * pi * r.x);
        double rz = sqrt(1.0 - r.y);
        vector_3 rr = vector_3(uu * rx + vv * ry + normal * rz);

        return rr.normalize();
}
```

Finally, the intersection test functions are:

```
real_type intersect_AABB(
        const vector_3 min_location,
        const vector_3 max_location,
        const vector_3 ray_origin,
        const vector_3 ray_dir,
        real_type& tmin,
        real_type& tmax)
{
        tmin = (min_location.x - ray_origin.x) / ray_dir.x;
        tmax = (max_location.x - ray_origin.x) / ray_dir.x;

        if (tmin > tmax)
                swap(tmin, tmax);

        real_type tymin = (min_location.y - ray_origin.y) / ray_dir.y;
        real_type tymax = (max_location.y - ray_origin.y) / ray_dir.y;

        if (tymin > tymax)
                swap(tymin, tymax);

        if ((tmin > tymax) || (tymin > tmax))
                return 0;

        if (tymin > tmin)
                tmin = tymin;

        if (tymax < tmax)
                tmax = tymax;

        real_type tzmin = (min_location.z - ray_origin.z) / ray_dir.z;
        real_type tzmax = (max_location.z - ray_origin.z) / ray_dir.z;

        if (tzmin > tzmax)
                swap(tzmin, tzmax);

        if ((tmin > tzmax) || (tzmin > tmax))
                return 0;

        if (tzmin > tmin)
                tmin = tzmin;

        if (tzmax < tmax)
                tmax = tzmax;

        if (tmin < 0 || tmax < 0)
                return 0;

        vector_3 ray_hit_start = ray_origin;
        ray_hit_start.x += ray_dir.x * tmin;
        ray_hit_start.y += ray_dir.y * tmin;
```

```
        ray_hit_start.z += ray_dir.z * tmin;

        vector_3 ray_hit_end = ray_origin;
        ray_hit_end.x += ray_dir.x * tmax;
        ray_hit_end.y += ray_dir.y * tmax;
        ray_hit_end.z += ray_dir.z * tmax;

        real_type l = (ray_hit_end - ray_hit_start).length();

        return l;
}

real_type intersect(
        const vector_3 location,
        const vector_3 normal,
        const real_type receiver_distance,
        const real_type receiver_radius)
{
        const vector_3 circle_origin(receiver_distance, 0, 0);

        if (normal.dot(circle_origin) <= 0)
                return 0.0;

        vector_3 min_location(
                -receiver_radius + receiver_distance,
                -receiver_radius,
                -receiver_radius);

        vector_3 max_location(
                receiver_radius + receiver_distance,
                receiver_radius,
                receiver_radius);

        real_type tmin = 0, tmax = 0;

        return intersect_AABB(
                min_location,
                max_location,
                location,
                normal,
                tmin,
                tmax);
}
```

6 Results

6.1 Attractive gravitation

See Fig. 3 for a plot showing the strength of the Schwarzschildian and Newtonian gravitation. Three curves are shown: the numerical Schwarzschild gravitation produced by the field line code, the analytical Schwarzschild acceleration of Section 3, and the analytical Newtonian acceleration M_e/R^2. Several features of the plot deserve comment.

First, the numerical curve tracks the analytical Schwarzschild curve closely across the entire sampled range, from $R \approx 1.5 \times 10^5$ (close to the horizon, where the acceleration is largest) out to $R \approx 3 \times 10^5$. The small, jagged departures of the numerical curve from the smooth analytical curve are the expected shot noise: the intersection counts are finite, and the finite difference amplifies their fluctuations. The noise diminishes as the field line count n_e is increased, at the usual Monte Carlo rate of one over the square root of the count.

Second, at close proximity the Schwarzschild curves rise well above the Newtonian curve. This is the near-field enhancement discussed in Section 2: cosine weighted emission concentrates more intersecting line segments (and, more importantly, a steeper density gradient) near the emitter than normal emission does. An observer using Newtonian theory to model this regime would underestimate the acceleration by a large factor.

Third, at far proximity, the three curves converge. This is the weak-field limit: cosine weighted emission and normal emission produce the same far-field flux, and so the same far-field gradient. The model thus passes the essential consistency test – it deviates from Newton only where general relativity says it must.

6.2 Repulsive gravitation

Experimentation with the receiver size is necessary to understand the intricacies of the code. For instance, where $r_r = r_e \times 0.01$, it is found that gravitation can be repulsive in the far (but not too far) field. The mechanism is illustrated in Fig. 2: with non-normal emission, some field lines cross the x-axis obliquely, and at certain distances the far receiver box B captures *more* intersecting line segments than the near box A. The finite difference α then becomes positive, and the inferred acceleration points away from the emitter. The repulsion does not extend to future null infinity, because gravitation becomes Newtonian in that far-field regime. It should be noted that repulsive gravitation plagues all non-normal (e.g. cosine weighted) field line theories. Whether this repulsion is a genuine physical prediction, a discretization artifact, or a hint of something akin to dark energy at intermediate scales, is left as an open question; here we confine ourselves to characterizing it numerically.

See Fig. 4 for a plot of the onset of repulsive gravitation mean distance d based on field line count n, where $d \approx 0.01845 \times n^{0.723}$. The standard deviations also form a power law, where $s \approx 0.0486 \times n^{0.614}$. The power law fits were obtained by measuring the onset distance across a range of field line counts spanning several orders of magnitude, with multiple pseudorandom seeds per count; the error bars in Fig. 4 show the seed-to-seed variation. Because the exponent 0.723 is less than one, the onset distance grows sublinearly in the field line count: doubling the entropy of the emitter pushes the repulsive zone outward, but by less than a factor of two.

To give the power laws physical scale: the onset distance for an Earth-mass Schwarzschild black hole is $1.03 \pm 2.12\text{e}{-7}$ astronomical units (AU). That is, for an emitter with the mass of the Earth (and hence a Schwarzschild radius of roughly nine millimetres, and an entropy of roughly 10^{66} bits), the repulsive zone begins at approximately the Earth-Sun distance, with a remarkably small standard deviation. The smallness of the standard deviation is itself a consequence of the enormous field line count – the shot noise is beaten down by the sheer number of samples.

6.3 Practical notes on convergence

A few practical remarks for the programmer who wishes to reproduce these results. The finite difference step $\epsilon = 2r_r$ couples the spatial resolution to the receiver size: a smaller receiver gives a more local density estimate, but captures fewer field lines, and thus produces a noisier gradient. For a fixed field line budget, there is an optimal receiver size that balances discretization error against shot noise, and the constant 0.01 used here sits comfortably in that regime for $n_e = 1\text{e}11$. When in doubt, run the Newtonian configuration A) first, and tune the receiver size until the numerical curve sits on top of M_e/R^2; the same settings will then serve for configurations B) and C). It is also advisable to use double precision (or better) for the accumulators, since the intersection counts can grow large enough for single precision to lose low-order bits.

We leave the Kerr metric for a future tutorial.

7 Discussion and future work

The main lesson of this paper is that a remarkably simple statistical model – random field lines, an AABB receiver, and a finite difference – reproduces both the Newtonian inverse square law and the Schwarzschild near-field enhancement, with the switch between the two effected solely by the emission direction statistics. The chord-based strategy C) is particularly satisfying, since the cosine weighting is not put in by hand but emerges from the geometry of the horizon itself. In this sense, the model is a small, concrete, computable illustration of the holographic idea that the physics of gravitation is encoded on a bounding surface.

Several directions for future work suggest themselves:

- *The Kerr metric.* Rotation breaks spherical symmetry, and the emitter becomes oblate. A field line model of the Kerr solution would require an anisotropic emission law, and would allow a numerical study of frame dragging. As noted above, we leave this for a future tutorial.

- *The repulsive zone.* The power laws of Section 6 characterize the onset of repulsion, but the depth and width of the repulsive zone remain to be mapped in detail, as does its dependence on the receiver geometry (e.g. sphere versus AABB versus oriented bounding box).

- *GPU acceleration.* The inner loop – generate two random vectors, subtract, normalize, ray-AABB intersection test – is an ideal candidate for GPU compute shaders, which would push the practical field line count up by several orders of magnitude and shrink the shot noise accordingly.

- *Other receivers.* The AABB was chosen for speed and familiarity, but the density gradient can in principle be estimated with any convex receiver. It would be worthwhile to confirm that the results are independent of the receiver shape in the small-receiver limit.

- *Time-dependent fields.* The present model is static. Allowing the emitter to move, and assigning the field lines a finite propagation speed, would open the door to a field line treatment of gravitational radiation.

It is hoped that the tutorial style of this paper, and the availability of the complete C++ source code, will lower the barrier to entry for programmers who are curious about general relativity but who are put off by the formal apparatus of differential geometry. There is, after all, no substitute for computing something yourself.

8 Conflict of interest

There are no conflicts of interest to report.

9 Data availability

The C++ code is freely available, providing a way to reproduce the data.

References

[1] Halayka. Newtonian gravitation from scratch, for C++ programmers. (2024)

[2] ‘t Hooft. Dimensional reduction in quantum gravity. (1993)

[3] Misner et al. Gravitation. (1970)

Figure 1: This figure shows an axis-aligned bounding box and an isotropic emitter, looking from slightly above. An example field line (red) and intersecting line segment (green) are given. The bounding box is filled with these green intersecting line segments. It is the gradient of the density of these line segments that forms the gravitational acceleration. Note that the field line is normal to the surface of the emitter.

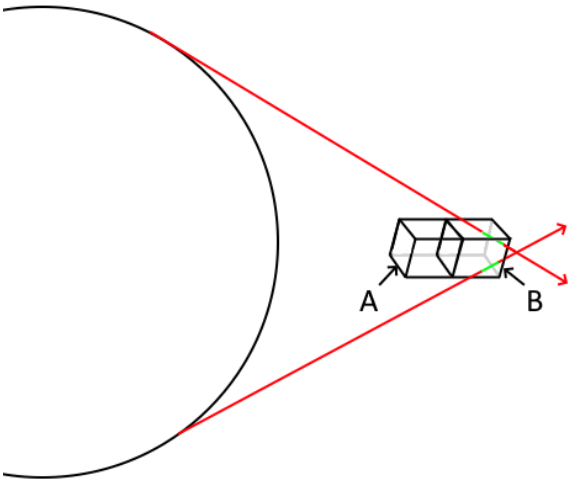

Figure 2: Axis-aligned bounding box A and box B are separated by the value epsilon $\epsilon = 2r_r$. Box B contains more field lines than box A, resulting in repulsive gravitation. Note that the field lines are not normal to the surface of the emitter.

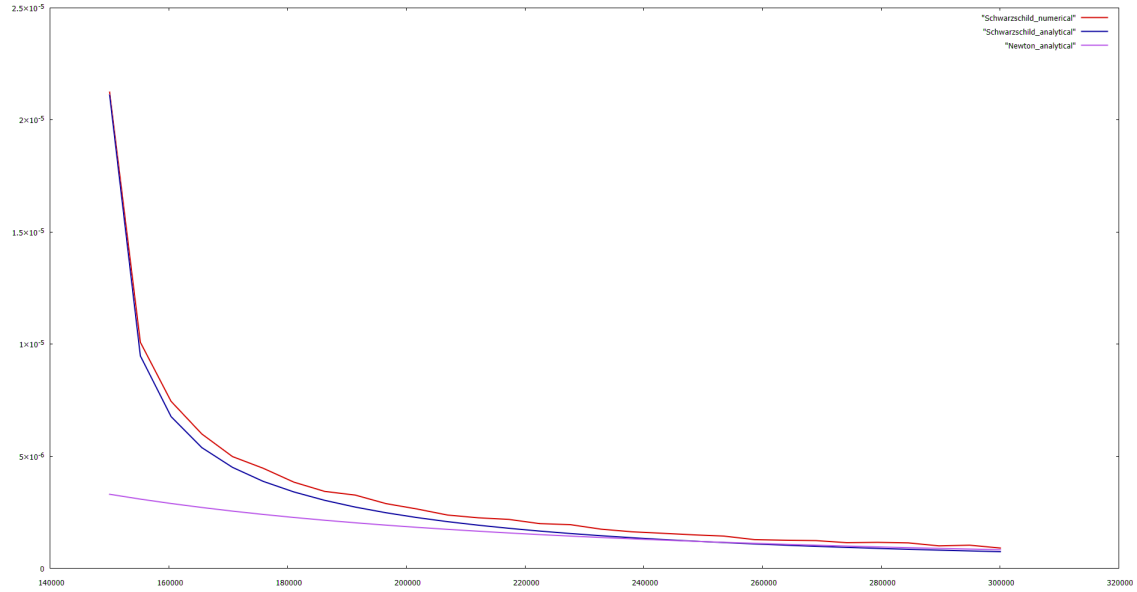

Figure 3: A plot showing the strength of the Schwarzschildian and Newtonian gravitation. The numerical Schwarzschild curve (red) tracks the analytical Schwarzschild curve (blue), and both converge to the analytical Newtonian curve (violet) in the far field.

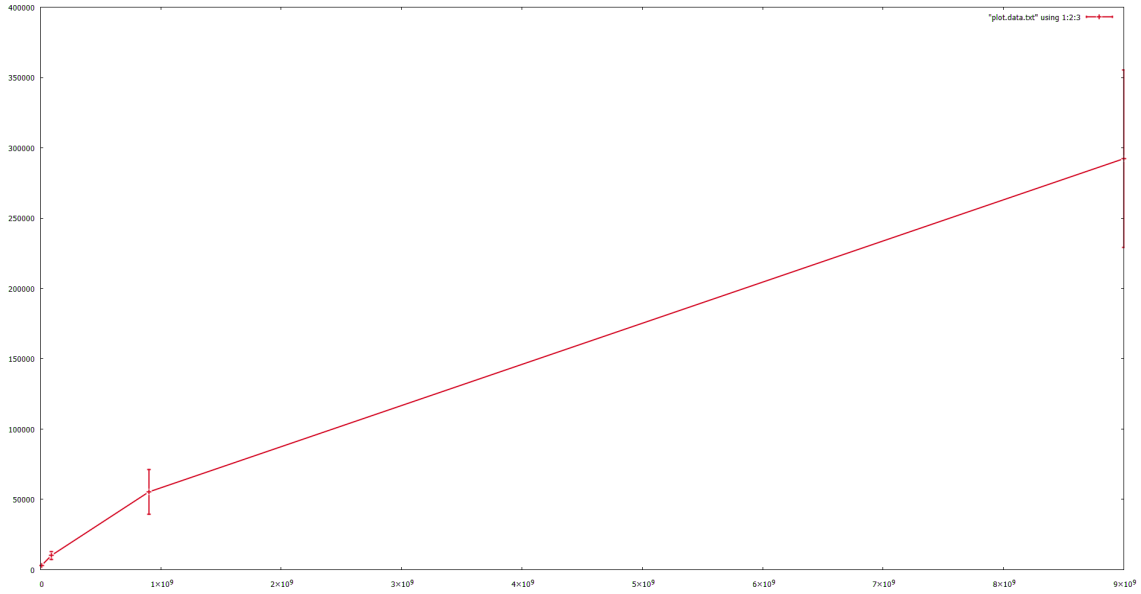

Figure 4: A plot of the onset of repulsive gravitation mean distance d based on field line count n. The error bars show the standard deviation across pseudorandom seeds.